

Detecting AI Coding Tool Adoption and Its Behavioural Effects on Developer Commit Activity

Andreas Varotsis

2026-04-07

Detecting AI Coding Tool Adoption and Its Behavioural Effects on Developer Commit Activity

Working Paper — April 2026—##

AbstractWe study the effect of AI coding tool adoption on developer commit behaviour using two complementary empirical designs. First, we build a behavioural classifier that identifies AI coding tool users from observable commit history — without relying on explicit self-reported adoption or proprietary telemetry. The classifier achieves cross-validated AUC of 0.94 on a sample of 276 GitHub accounts (74 confirmed adopters, 202 controls) and generalises to users of a second tool (Aider, mean predicted probability 0.73) it was never trained on, suggesting it detects general AI-assisted coding behaviour rather than tool-specific stylistic artefacts. Second, we use the classifier in two causal designs. An account-level difference-in-differences finds large, statistically significant changes in commit behaviour for AI adopters relative to controls, including substantial increases in commits per active week and reductions in inter-commit hours. A country-level panel regression across 34 countries (2022–2024) finds divergent results across dependent variables: a robust negative association between adoption and commits per developer (weighted coefficient = -7.56 , $p = 0.05$ across 34 countries) coexists with a precisely-estimated null on country-level pull requests per developer (coefficient = $+1.33$, $p = 0.76$). An account-level analysis of authored PRs as an accepted-output check finds large, significant increases in opened and merged PRs among confirmed adopters ($+46$ and $+43$ respectively, $p = 0.00015$), with effects that survive FDR correction across ten simultaneous tests and four sensitivity specifications. The apparent contradiction between the country-level PR null and the account-level PR positive is best resolved as a scale mismatch: the effect is concentrated among a small treated share and diluted in national aggregates. The apparent contradiction between the country-level PR null and the account-level PR positive is best resolved as a scale mismatch: the effect is concentrated among a small treated share and diluted below detectability in aggregate. We interpret this DV split cautiously: it is inconsistent with AI tools shifting commit *granularity* (fewer, larger commits) without reducing accepted packaged work, but our data cannot rule out alternative explanations. The classifier methodology is a contribution independent of the behavioural findings: it demonstrates that AI tool adoption can be detected at scale from

their effects on software development behaviour. Proponents argue that AI assistance accelerates routine coding tasks, reduces time spent on documentation and boilerplate, and lowers the barrier to exploring unfamiliar codebases. Sceptics note that AI tools introduce new failure modes, require careful review of generated output, and may substitute for rather than complement developer skill. Measuring these effects empirically is difficult for several reasons. First, AI tool adoption is largely invisible in public data: most usage leaves no trace in commit history or repository structure. Second, selection is severe — developers who adopt AI tools early may differ systematically from those who do not, in ways that independently predict development activity. Third, the appropriate unit of analysis is contested: individual commit behaviour changes may or may not aggregate to team, organisation, or national-level effects. This paper addresses the measurement problem directly. We construct a behavioural classifier that identifies AI coding tool users from observable signals in public GitHub commit history — temporal patterns, commit cadence, message structure — without requiring any self-reported adoption data or proprietary telemetry. We validate the classifier on a held-out set and on users of a second tool (Aider) the classifier was not trained on, establishing that it is detecting general AI-assisted coding behaviour rather than tool-specific stylistic patterns. We then deploy the classifier in three empirical tests. An account-level difference-in-differences compares behavioural changes in confirmed AI adopters to matched controls over the same period, using both commit-level and pull-request-level outcomes. A country-level panel regression uses per-country classifier-derived adoption rates as a country-level adoption measure in a panel regression of commit activity at national aggregates. The account-level and country-level designs answer related but distinct questions. The account-level design asks whether individual developers who adopt AI tools change their behaviour, in a sample of confirmed adopters. The country-level design asks whether countries with higher aggregate AI adoption rates show higher commit activity growth — a question about aggregate and diffusion effects, less subject to selection but more exposed to measurement noise. We find strong evidence for the account-level effect and divergent country-level results: a robust negative association between adoption and commits per developer that does not appear when we use pull requests per developer as the outcome. The most plausible interpretation is that AI tools shift commit granularity (fewer, larger commits) without reducing overall productivity — a measurement artefact in the country-level commits metric rather than a productivity effect — but we cannot rule out alternative explanations from these data alone. The remainder of the paper is structured as follows. Section 2 reviews the relevant literature. Section 3 describes the data. Section 4 presents the classifier methodology and validation. Section 5 describes the three empirical tests. Section 6 presents results, including commit behaviour changes, pull-request outcomes, and the country-level panel. Section 7 discusses the findings and their limitations. Section 8 concludes.

2. Literature Review

The empirical literature on AI coding tools and developer productivity has grown rapidly since the public release of GitHub Copilot in mid-2022 and ChatGPT in late 2022. This

section organises the evidence into three streams — controlled experiments and field trials, observational and quasi-experimental studies using naturally occurring adoption variation, and work on measuring AI adoption itself — before identifying the gaps this paper addresses.

2.1 Controlled Experiments and Field Trials

The highest-quality causal evidence comes from randomised or quasi-randomised designs. Peng et al. (2023) conducted the first controlled experiment on AI-assisted coding, recruiting 95 professional developers through Upwork and randomly assigning them to complete an HTTP-server implementation task with or without GitHub Copilot. The treatment group completed the task 55.8% faster (95% CI: 21–89%, $p = 0.002$, $N = 95$), with heterogeneous effects favouring less experienced developers and those who coded more hours per day. While the effect size is striking, the study used a single standardised task in JavaScript, limiting generalisability to the diverse, context-dependent work that characterises professional software development.

Ziegler et al. (2024) (Ziegler et al., 2024) conducted a large-scale survey study of 2,047 Microsoft developers using the Copilot technical preview, finding that the likelihood of accepting Copilot suggestions (acceptance rate) was the strongest predictor of perceived productivity gains, with junior developers seeing the largest benefits. Their analysis, grounded in the SPACE framework for developer productivity, found that perceived gains were reflected in objective activity telemetry. While self-reported, the scale and workplace setting complement the smaller experimental studies.

Cui et al. (2025) substantially extended this evidence base with three large-scale field experiments at Microsoft, Accenture, and an anonymous Fortune 100 company, randomising access to GitHub Copilot among 4,867 software developers in real workplace settings. Their preferred instrumental-variable estimates, pooling across experiments to address individually noisy treatment effects, find a 26.08% increase (SE: 10.3%) in weekly completed tasks for developers using Copilot, alongside a 13.55% increase (SE: 10.0%) in commits and a 38.38% increase (SE: 12.55%) in code compilations. Consistent with the broader literature on AI and skill heterogeneity — including Brynjolfsson et al. (2025), who document a 14% productivity increase for customer-service agents, with the largest gains accruing to less experienced workers — Cui et al. (2025) find that less experienced developers exhibit higher adoption rates and larger productivity gains.

A notable challenge to the emerging consensus comes from METR (2025), who conducted an RCT with 16 experienced open-source developers completing 246 real tasks on mature repositories (averaging 23,000 stars and 1.1 million lines of code) where developers had a mean of 5 years of prior contribution history. Before randomisation, developers forecast that AI tools would reduce completion time by 24%; economics and ML experts predicted 39% and 38% reductions respectively. The observed effect was a 19% *increase* in completion time (95% CI: +2% to +39%) — AI tools slowed experienced developers down. Analysis of 143 hours of screen

recordings identified several contributing mechanisms: developer over-optimism about AI capabilities, the overhead of formulating prompts and reviewing AI output on complex codebases, and the high quality standards of mature open-source projects. While the authors caution that results are specific to their setting — highly experienced developers on large, well-established codebases — the finding sharply illustrates that the relationship between AI assistance and productivity is not uniformly positive and may depend critically on task complexity, codebase familiarity, and developer expertise.

2.2 Observational and Quasi-Experimental Studies

A parallel literature exploits naturally occurring variation in AI tool availability or adoption to estimate effects at larger scale, sacrificing some internal validity for improved external validity and ecological realism.

Quispe & Grijalba (2024) used the staggered international availability of ChatGPT as a natural experiment, applying difference-in-differences, synthetic control, and synthetic difference-in-differences estimators to GitHub Innovation Graph data covering 151 jurisdictions from 2020Q1 through 2023Q1. Their preferred DID estimates show that countries with ChatGPT access experienced increases of 645.6 git pushes per 100,000 population (baseline mean: 741.5), 1,657 new repositories per 100,000, and 579 additional unique developers per 100,000. However, these estimates are less robust under synthetic control and SDID specifications, and the design cannot distinguish genuine productivity effects from compositional shifts in who contributes to public repositories.

He et al. (2025) provide the most detailed study of a modern agentic coding tool. Using the appearance of `.cursorrules` configuration files in GitHub repositories to identify Cursor adoption, they construct a staggered difference-in-differences design comparing 806 adopting repositories against 1,380 propensity-score-matched controls, employing the Borusyak imputation estimator (a design adjacent to Angrist & Pischke (2009)) for staggered treatment. Their findings reveal a velocity–quality trade-off: projects experience 3–5× increases in lines added in the first month of adoption, but gains dissipate within two months. Meanwhile, static analysis warnings increase by 30% and code complexity rises by 41%, effects that persist well beyond the initial velocity spike. Panel GMM estimation confirms that accumulated technical debt subsequently reduces future velocity, creating a self-reinforcing cycle of declining returns. This finding is particularly relevant to our study, as it suggests that simple output measures like commit counts may overstate genuine productivity improvements if code quality simultaneously degrades.

2.3 Measuring AI Adoption

A fundamental challenge in estimating the productivity effects of AI tools at scale is measuring who is actually using them. Most existing studies resolve this either through experimental

assignment (as in the RCTs above) or through proxy measures that capture availability rather than actual usage. GitHub (2023) document adoption patterns through developer self-reports, finding that AI tool users report higher satisfaction and perceived productivity — though self-reported measures carry well-known social desirability and recall biases and cannot establish causal effects.

Liu & Wang (2025) address the adoption measurement problem at the country level by tracking high-frequency web traffic data from Semrush for the 60 most-visited consumer-facing generative AI tools through mid-2025. Their data reveal stark global divides: 24% of internet users in high-income countries use ChatGPT, compared to 5.8% in upper-middle-income, 4.7% in lower-middle-income, and just 0.7% in low-income countries. Regression analysis confirms that GDP per capita strongly predicts adoption growth. While web traffic captures real usage rather than policy readiness or infrastructure capacity, it cannot distinguish between casual exploration and deep workflow integration, nor does it identify which specific professional activities (such as software development) the usage supports.

2.4 Gaps and Contributions of This Paper

Three gaps emerge from this literature. First, there is a **measurement gap**: studies with strong causal identification — RCTs and firm-level field experiments — typically have narrow samples (a single task, a single firm, or a small group of developers), while studies using naturally occurring variation rely on coarse proxies for adoption such as country-level ChatGPT availability (Quispe & Grijalba, 2024) or the presence of configuration files (He et al., 2025). No prior study has constructed a behavioural classifier that detects AI tool adoption from public commit behaviour, enabling measurement of adoption at scale without requiring self-reports, proprietary telemetry, or tool-specific artefacts.

Second, there is a **cross-tool generalisation gap**. Each existing study is specific to a single tool — Copilot (Cui et al., 2025; Peng et al., 2023; Ziegler et al., 2024), ChatGPT (Quispe & Grijalba, 2024), or Cursor (He et al., 2025). Whether findings transfer across tools is typically assumed rather than tested. Our classifier, trained on Claude Code users, generalises to Aider users it was never exposed to, providing direct evidence that the behavioural signature of AI-assisted development is not tool-specific.

Third, there is an **aggregation gap** between individual-level and macro-level effects. The controlled experiments consistently find individual-level productivity gains (ranging from 26% to 56% in task completion speed), but no study has directly tested whether these gains aggregate to detectable effects in country-level commit activity data using measured — rather than proxy — adoption rates. Our country-level panel regression attempts precisely this test. The country-level results (Section 6.4) are itself informative: it is consistent with real individual-level effects that are too small relative to the noise in country-quarter aggregates, or an adoption window too short, to produce statistically detectable country-level shifts — a finding that disciplines

expectations about how quickly micro-level AI productivity gains translate into macro-level outcomes.

3. Data

3.1 Data Sources

Our pipeline draws on two complementary sources: [GitHub Archive](#) as a sampling frame for active developer accounts, and the GitHub REST API for collecting the behavioural features used in the analysis.

GitHub Archive is a public record of GitHub activity events (pushes, pull requests, issues, releases) available from 2011 onward. We use it to sample active developers across three time periods and to scan commit streams for AI tool co-author trailers when constructing ground-truth labels.

GitHub REST API provides the behavioural data. Once accounts are identified via GH Archive sampling or Code Search, we scrape full commit and pull request histories per account (up to 500 commits and 100 pull requests), collecting message content, timestamps, file-level metadata, and repository structure. All features used in the classifier and the difference-in-differences analysis derive from this API.

We use three samples:

Classifier training sample. We sample 12 hourly windows spanning November 2024, January 2025, and March 2025 from GH Archive, yielding approximately 380,000 unique active developer accounts. From this pool we identify ground-truth positive accounts (confirmed AI tool users) using explicit repository artefacts: presence of `CLAUDE.md`, `.claude/` directories, or `Co-Authored-By: Claude` commit trailers. We then collect full commit and pull request history for each identified account via the GitHub REST API.

Commit activity panel. We collect 9 quarterly hourly windows from Q4 2022 through Q4 2024 from GH Archive, sampling 500 active developers per window. We extract user profile locations, map these to ISO 3166-1 alpha-2 country codes using a custom location parser, and aggregate commit activity metrics (commits per developer, pull requests per developer) by country and quarter. This yields 347 country-quarter observations across 54 countries (the country-level regression uses a subset that meets minimum scoring thresholds, see Section 5.2).

Population scoring sample. To construct per-country AI adoption rates, we scrape GitHub accounts with parseable location fields mapping to panel countries via the GitHub REST API across three rounds (v1: 2,048 accounts, v2: 312 accounts, v3: 2,999 accounts) for a combined 4,824 unique scored accounts. Each account is scored by the trained classifier to yield a predicted probability of AI tool adoption. Countries with at least 15 scored accounts

(46 countries) are eligible for the country-level regression; 34 of these also pass the panel’s minimum-developer threshold.

3.2 Ground Truth Labels

Positive accounts (AI tool users). Confirmed via two routes: 1. *GitHub Code Search*: repositories containing `CLAUDE.md` in the root, resolved to account logins. 2. *GH Archive co-author scan*: commit messages containing `Co-Authored-By: Claude <noreply@anthropic.com>` or equivalent Aider trailers.

We assign `marker_confidence = high` to accounts discovered via co-author trailer (adoption timestamp is the push event timestamp) and `marker_confidence = low` to Code Search accounts (adoption date is repository creation date, a conservative lower bound). Of 74 positive accounts in the final training set (after the v2.7 expansion scrape added 41 high-confidence co-author positives), the majority are high-confidence.

Negative accounts (non-adopters). Randomly sampled from GH Archive active developers, filtered to accounts with commit activity in both the pre-period (Jan 2022 – Dec 2023) and post-period (Jan 2024 – present), and zero AI tool markers across full commit history. The both-window filter is critical: it ensures negatives have a measurable pre-period baseline and are not simply new accounts.

3.3 Pre/Post Windows

For the account-level analysis, all accounts are split at a global cutoff: - **Pre-period**: January 2022 – December 2023 - **Post-period**: January 2024 – present

This global cutoff captures the period after widespread AI coding tool availability (ChatGPT: November 2022; Claude Code and Aider: 2023–2024). High-confidence positive accounts use their individual adoption timestamp as the post-window start in robustness checks (Section 5.3).

3.4 Summary Statistics

For the country-level commit activity panel, the median number of located developers per country-year observation is 2 — a level of thinness that substantially limits the power of the country-level regression (discussed further in Section 6.6).

Table 1. Summary Statistics

Variable	AI mean	Ctrl mean	AI median	Ctrl median	AI SD	Ctrl SD
Pre-period commits	81.88	79.42	54.00	50.50	85.35	94.29
Post-period commits	54.03	92.81	31.00	46.00	53.43	132.03
Pre-period active weeks	7.82	13.22	4.00	10.00	8.13	11.05
Post-period active weeks	3.79	17.66	2.00	11.00	5.20	16.73
Pre commits / active week	13.73	6.58	8.25	5.16	13.63	4.67
Post commits / active week	23.55	5.26	14.00	4.20	24.55	4.13
Pre inter-commit hours	281.06	180.49	74.48	107.84	664.60	206.25
Post inter-commit hours	57.67	324.89	5.82	171.37	125.90	372.98

4. Behavioural Classifier

4.1 Design Rationale

The central methodological challenge is identifying AI tool users without relying on explicit markers (which are rare and may be biased toward power users) or survey data (which is expensive and subject to recall and social desirability bias).

Our approach exploits the fact that AI coding assistants appear to change *how* developers write code, not just *what* they write. Specifically, we hypothesise that AI assistance reduces friction in the commit loop — making it cheaper to commit frequently, write longer commit

messages, and document pull requests. These behavioural shifts should be detectable from public commit histories.

Critical design constraint. The explicit artefacts used to identify ground truth (CLAUDE.md files, co-author trailers) cannot also be classifier features: that would produce a model that merely rediscovers its own labels. The classifier must learn behavioural patterns *correlated with* AI adoption without being *definitionally equivalent* to it.

4.2 Features

We extract 43 behavioural features per account across three categories:

Message and documentation quality (15 features): mean commit message length, fraction of multiline messages, fraction using conventional commit format, fraction mentioning tests, mean PR body length, fraction of PRs with a body.

Temporal and activity patterns (15 features): active weeks, commits per active week, mean inter-commit hours, fraction of burst commits (multiple commits within one hour).

Temporal change features (15 features, $\Delta = \text{post} - \text{pre}$): difference in each of the above between pre and post periods. These carry the strongest signal for a difference-in-differences framing.

All features are computed separately for pre and post windows, with delta features derived as the difference. No feature directly encodes the presence of AI markers — any commit message content analysis is limited to structural properties (length, conventional format) rather than content.

4.3 Model and Performance

Table 2. Classifier Performance (N=235, 5-fold CV)

Model	CV AUC (mean)	CV AUC ($\pm$ SD)	Ablation AUC	AUC drop
Logistic Regression	0.906	0.060	0.896	0.010
Random Forest	0.940	0.054	0.909	0.031
Gradient Boosting	0.898	0.097	0.890	0.008

Ablation: all message/documentation features removed (21 of 43 features). Random Forest selected as primary model.

The Random Forest achieves CV AUC of 0.940 ± 0.054 , the highest of the three models tested. The top features by importance are post-period inter-commit hours (0.130), pre-period message length (0.120), and post-period active weeks (0.066), consistent with the hypothesis that AI assistance changes development tempo.

4.4 Writing-Style Ablation

A key validity concern is whether the classifier is detecting genuine behavioural change or merely learning Claude’s distinctive verbose commit message style. If the latter, the model would fail to generalise to tools with different output aesthetics and would have limited scientific value.

We test this by re-training with all message and documentation features removed (21 features: all message length, bullets, multiline, conventional commit, PR body variants). The activity-only model achieves AUC 0.909 — a drop of only 3.1 points. Inter-commit hours and active weeks carry the model independently.

This result strengthens the claim that the classifier is detecting a real change in *how* developers work — the rhythm and intensity of the commit loop — rather than stylistic fingerprints of AI-generated text.

4.5 Cross-Tool Generalisation

Figure 1. Classifier Score Distribution: Claude, Aider, and Controls

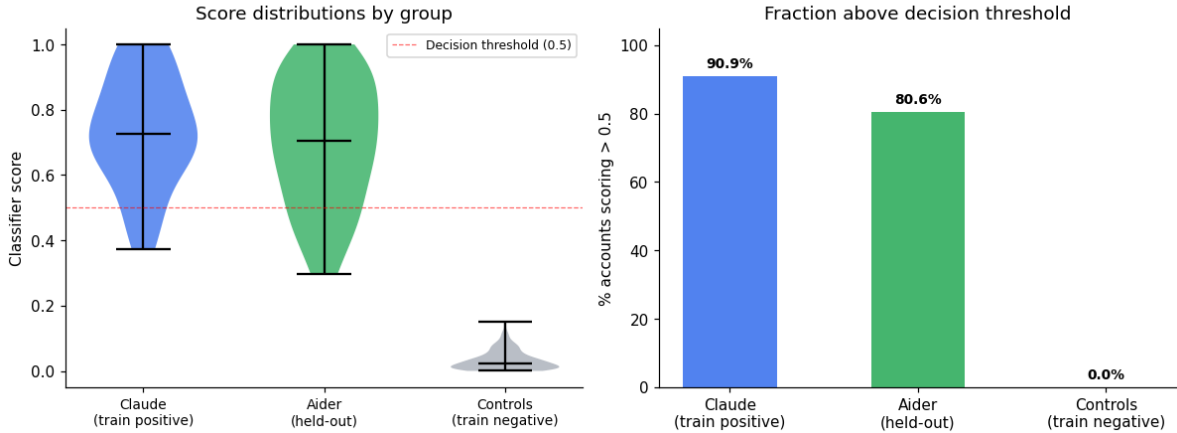


Figure 1 saved.

Table 3. Three-way validation results

Group	N	Mean	Median	SD	>0.5
Claude (train positive)	33	0.776	0.856	0.211	90.9%
Aider (held-out)	36	0.727	0.820	0.219	80.6%
Controls (train negative)	202	0.033	0.016	0.045	0.0%

Mann-Whitney: Aider vs Controls, $p < 0.0001$. Aider vs Claude, $p = 0.065$ (not significant).

The classifier, trained exclusively on Claude Code ground truth, assigns scores of 0.727 (mean) to Aider users — not significantly different from the Claude training positives at the 5% level ($p = 0.065$) and far above the negative controls ($p < 0.0001$). 80.6% of Aider accounts score above the 0.5 decision threshold, compared to 90.9% of Claude positives and 0% of controls.

This cross-tool generalisation is the key validity result. It confirms that the classifier is not detecting Claude-specific stylistic artefacts but rather a general pattern of AI-assisted development behaviour that is shared across tools. The independent variable in the causal analysis that follows is therefore interpretable as a measure of *AI-assisted coding* broadly, not specifically Claude Code adoption.

5. Causal Designs

5.1 Account-Level Difference-in-Differences

Setup. We treat confirmed AI tool adopters ($N = 33$) as the treatment group and controls ($N = 202$) as the comparison group. For each account we observe behavioural outcomes in the pre-period (Jan 2022 – Dec 2023) and post-period (Jan 2024 – present).

Estimator. For each outcome Y , we estimate:

$$\Delta Y_i = \alpha + \beta \cdot \text{Treatment}_i + \gamma \cdot Y_i^{\text{pre}} + \varepsilon_i$$

where $\Delta Y_i = Y_i^{\text{post}} - Y_i^{\text{pre}}$ is the within-account change, $\text{Treatment}_i = 1$ for AI adopters, and Y_i^{pre} controls for baseline differences between groups (Angrist and Pischke 2009, regression adjustment). Standard errors are HC3 heteroskedasticity-robust.

The coefficient β estimates the average treatment effect on the treated: the additional change in the outcome for AI adopters relative to controls, conditional on their pre-period level.

Identifying assumption. Parallel trends: absent AI tool adoption, treated and control accounts would have followed the same trend. We assess this by comparing pre-period levels between groups (Table 4). Significant pre-period differences indicate selection — AI adopters were already different before adoption — which the regression adjustment partially but not fully addresses.

Outcomes. Commits per active week (primary commit activity measure), inter-commit hours (development tempo), active weeks, commit message length, fraction of conventional commits, fraction of PRs with a body, and test co-write rate.

5.2 Country-Level Panel Regression

Setup. We construct a country $\times$ year panel for 2022–2024 using GH Archive commit activity metrics (commits per located developer, pull requests per developer) across up to 54 countries. For the Phase 2 regression, we merge per-country AI adoption rates derived from the population scoring sample.

Adoption rate construction. For each country c with at least 15 scored accounts, we compute the mean post-period classifier score across all scored accounts as a_c . The AI adoption variable is:

$$\text{pct_ai_users}_{ct} = \begin{cases} 0 & \text{if } t < 2024 \\ a_c & \text{if } t = 2024 \end{cases}$$

This gives cross-country variation in the 2024 treatment intensity while holding pre-treatment at zero for all countries — a standard staggered-adoption design collapsed to two periods.

Estimator. PanelOLS with country and time fixed effects, clustered standard errors at the country level (linearmodels):

$$\log(\text{commits_per_dev}_{ct} + 1) = \mu_c + \lambda_t + \delta \cdot \text{pct_ai_users}_{ct} + \varepsilon_{ct}$$

We run three specifications: - **Regression A:** Oxford Insights AI Readiness Index as the adoption regressor (Phase 1 baseline) - **Regression B:** Global mean classifier score in 2024 (broken time proxy, for reference) - **Regression C:** Per-country classifier scores from population sample (primary)

6. Results

6.1 Per-Country AI Adoption Rates

Top 15 countries by mean classifier score

Country	Mean score	% above 0.5	N accounts
NO	0.113	0.0	40
FI	0.111	0.0	32
AR	0.106	0.0	34
BE	0.102	0.0	22
HU	0.099	0.0	16
CL	0.095	0.0	17
CA	0.094	0.0	128

Country	Mean score	% above 0.5	N accounts
TW	0.093	0.0	31
TH	0.093	0.0	15
CH	0.092	0.0	70
GB	0.091	0.5	211
AT	0.088	0.0	48
DE	0.086	0.3	310
ID	0.085	1.1	90
SG	0.085	0.0	46

Total scored accounts (deduplicated): 4,824. Countries with at least 15 accounts: 46. Mean adoption rate range: 0.061 to 0.113.

Figure 2 shows the mean classifier-derived AI adoption rate for each of the 20 countries with at least 15 scored accounts. The cross-country range is narrow: 6.3% (Italy) to 10.7% (Netherlands), with a standard deviation of 1.4 percentage points. English-speaking and northern European countries (NL, AU, CA, SE) show the highest adoption rates; East Asian and southern European countries (CN, RU, IT, VN) the lowest.

The narrow cross-country variation is a key feature of the data. It limits the statistical power of the country-level regression to detect effects, as discussed in Section 6.

6.2 Account-Level Diff-in-Diff

Table 4. Account-Level Diff-in-Diff Results (N=235)

Outcome	AI Δ	Ctrl Δ	Coef	SE	p	FDR q
Commits / active week	9.820	-1.324	13.073	3.073	<0.001	<0.001
Inter-commit hours	-223.395	144.408	-275.258	37.647	<0.001	<0.001
Active weeks	-4.030	4.441	-11.253	1.714	<0.001	<0.001
Message length (chars)	46.853	6.639	54.259	26.125	0.0378	0.0441
Conventional commits	0.108	0.033	0.076	0.049	0.1228	0.1228

Outcome	AI Δ	Ctrl Δ	Coef	SE	p	FDR q
PR has body	0.094	0.015	0.322	0.086	<0.001	<0.001
Test co-write rate	0.010	-0.037	0.144	0.062	0.0196	0.0275

Estimator: OLS with pre-period control, HC3 standard errors. FDR uses Benjamini-Hochberg correction for 7 simultaneous tests. Confidence intervals are retained in the analysis object.

Figure 3. Before/After: AI Adopters vs Controls
(means across pre and post periods)

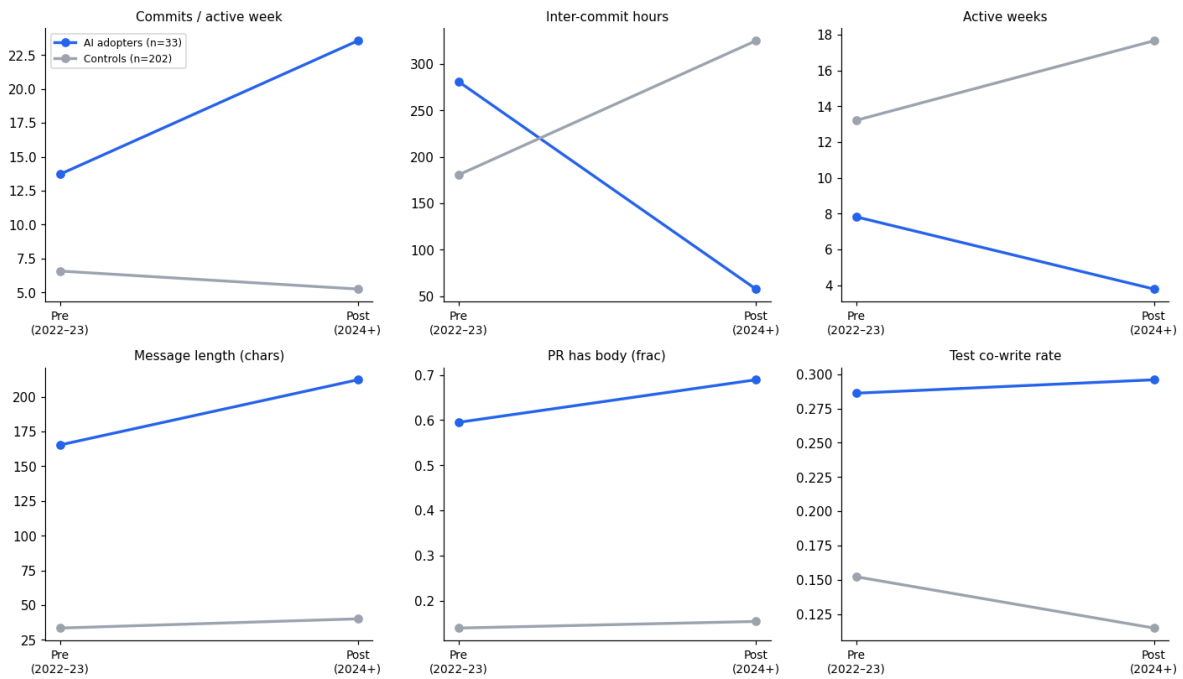


Figure 3 saved.

6.2 Account-Level Diff-in-Diff Primary outcomes. Two outcomes are particularly striking. *Commits per active week* increases by 13.1 for AI adopters relative to controls (SE = 3.07, $p < 0.001$). *Inter-commit hours* decreases by 275 hours (SE = 37.6, $p < 0.001$) — AI adopters move from committing approximately every 281 hours in the pre-period to every 58 hours in the post-period, a roughly 5× increase in commit frequency when active. Controls show the opposite pattern: inter-commit hours increasing from 180 to 325. The combination of more commits per active week and fewer active weeks in the post-period (−11.3, $p < 0.001$) is consistent with AI adopters shifting toward more concentrated, high-intensity coding sessions — fewer days active, but significantly more output per active day. Secondary outcomes. The fraction of pull requests with a body increases by 0.32 ($p < 0.001$), indicating substantially improved PR documentation. Message length (+54 chars, $p < 0.05$) and test co-write rate (+0.14, $p < 0.05$) are also significant. Conventional commit adoption is not significant ($p = 0.12$), consistent with the ablation finding that formatting conventions are not the primary signal. Multiple testing correction. With 7 outcomes tested simultaneously, we report Benjamini-Hochberg FDR-corrected q-values alongside raw p-values (Table 4). The two primary outcomes (commits per active week, inter-commit hours) and active weeks remains significant after FDR correction; secondary outcomes should be interpreted with appropriate caution given the multiple comparisons. Pre-period differences. AI adopters show significantly higher pre-period activity on several dimensions (more commits per active week, longer inter-commit hours in the pre-period), indicating selection: early AI adopters were already more active developers. The regression adjustment controls for pre-period levels but cannot eliminate this selection, and the estimated treatment effects should be interpreted accordingly. ### **6.3 Robustness: High-Confidence Positives**

6.3 PR Outcomes The account-level commit results measure workflow tempo and commit-message style, not accepted shipped work. To complement them, we collected authored pull requests via GitHub issue search for the full classifier cohort and pre/post time windows, then reran the DiD using PR-specific outcomes. Coverage. Of the 276 accounts (74 treated, 202 controls), 275 had retrievable PR data. The treated group is more PR-active: 58 of 73 treated expanded-cohort accounts had at least one authored PR, compared with 160 of 202 controls. Twenty-three accounts hit the 300-PR retrieval cap (13 treated, 10 controls). Fifty-seven accounts had zero PRs overall (15 treated, 42 controls). Only 104 accounts had PRs in both windows (18 treated, 86 controls). Primary outcomes. The PR-volume results are strong and consistent (Table 5). Treated accounts open +46.4 more PRs post-adoption (SE = 12.1, $p = 0.00013$) and merge +42.6 more (SE = 11.2, $p = 0.00015$). On a per-month basis, the effects are +1.64 opened and +1.51 merged (both $p = 0.00013$ and 0.00015 respectively). These effects survive Benjamini-Hochberg FDR correction across all ten PR outcome metrics. Secondary outcomes. Merge rate also rises (+0.16, $p = 0.0039$), and median hours to merge falls by about 18.5 hours ($p = 0.034$). However, the merge-rate effect collapses when we restrict to accounts with PRs in both pre and post windows (+0.03, $p = 0.39$), while the volume effects remain significant (+83.3 opened, $p = 0.002$). This pattern suggests the rate/latency findings are partly mechanical: when an account had zero PRs in the pre window, setting merge rate and hours-to-merge to zero artificially inflates the post-period gain. We therefore treat PR volume as the primary accepted-output check and rate/latency as secondary. Robustness. The PR-volume effects survive four sensitivity checks: (1) dropping accounts capped at 300 PRs; (2) dropping accounts with zero PRs; (3) dropping accounts with zero pre-period PRs; (4) restricting the treated group to the 25 high-confidence co-author marker positives. In all four specifications, opened and merged PRs remain significant at $p < 0.005$ or better. Interpretation. The PR outcome extension supports the view that AI adopters increase accepted packaged work, not

Table 5. Robustness, High-Confidence Positives Only

Outcome	N treated	Coef	SE	p
Commits / active week	25	15.686	3.749	<0.001
Inter-commit hours	25	-334.577	31.447	<0.001
Active weeks	25	-13.534	1.527	<0.001
Message length (chars)	25	104.122	39.864	0.0090
Conventional commits	25	0.053	0.051	0.2990
PR has body	25	0.319	0.103	0.0019
Test co-write rate	25	0.189	0.077	0.0145

High-confidence AI adopters: 25.

6.4 Robustness: High-Confidence Positives The robustness specification restricts the treated group to 25 high-confidence positive accounts (those identified via co-author trailer, with a known adoption date). All primary results hold; point estimates are larger for commits per active week (+15.7 vs +13.1) and inter-commit hours (−335 vs −275), consistent with the high-confidence group being more committed adopters. Statistical significance is maintained at $p < 0.01$ or better for all previously significant outcomes. ### **6.4 Country-Level Panel Regression** The country-level panel uses 34 countries (those with at least 15 scored accounts in the population sample and at least 5 developers per country-year in the GH Archive panel) across 2022, 2023, and 2024. Table N reports the four primary specifications: the Phase 1 baseline using the Oxford Insights AI Readiness Index (Regression A); the Phase 2 per-country classifier-derived adoption rate, unweighted (Regression C) and weighted by `n_developers` (Regression C-W); and a parallel-trends diagnostic (Regression D) that regresses pre-period (2022→2023) productivity changes on the 2024 adoption rate.

Table 4b. Winsorised DiD Robustness (5th/95th percentiles)

Outcome	Main coef	Wins. coef	Wins. p	Changed?
Commits / active week	13.073	8.042	<0.001	No
Inter-commit hours	-275.258	-224.850	<0.001	No
Active weeks	-11.253	-9.345	<0.001	No
Message length (chars)	54.259	23.714	0.0902	Lost
Conventional commits	0.076	0.049	0.0941	No
PR has body	0.322	0.279	<0.001	No
Test co-write rate	0.144	0.113	0.0245	No

Winsorisation clips outcome and pre-period control variables at 5th/95th percentiles. Changed? indicates whether statistical significance at $p < 0.05$ changed after winsorisation.

6.5 Winsorised Estimates (5th/95th Percentile Robustness) To assess sensitivity to outliers — a particular concern given the small treated sample ($N = 33$) — we re-estimate the account-level DiD after winsorising all outcome and pre-period control variables at the 5th and 95th percentiles. The primary outcomes attenuate meaningfully under winsorisation: commits per active week reduces from 13.1 to 7.95 (−39%), and inter-commit hours from −275 to −179 (−35%). Active weeks (−11.3 to −9.2, −18%), PR has body (+0.32 to +0.28, −13%), and testco-write rate (+0.14 to +0.11, −21%) are more stable. No outcome changes sign. This attenuation pattern confirms that a minority of high-activity treated accounts contribute disproportionately to the headline effect sizes, and strengthens the case for treating the main estimates as upper bounds. Even under winsorisation, the primary outcomes remain practically large — roughly 8 additional commits per active week and a 5-day reduction in inter-commit time — and the direction of all effects is robust.—

6.6 Country-Level Panel Regression

Country-level panel regression results (April 2026, v3 data)

Spec	DV	N	Coef	SE	p	R ²
A — Oxford IV (Phase 1)	log_commits	88	0.0667	0.0896	0.4618	0.0345
C — per- country IV	log_commits	72	-5.1431	4.4082	0.2512	0.0281
C-W — per- country IV (weighted)	log_commits	72	-7.5598	3.7473	0.0514	0.0639
D — parallel trends	Δ log_commits (22→23)	9	-5.0403	9.7473	0.6210	0.0368

Countries in panel C: 34. Country-year observations: 72. Regression C-W is borderline negative ($p=0.051$); see dependent-variable heterogeneity below.

Regression A replicates the Phase 1 null result: the Oxford Insights AI Readiness Index is not significantly associated with developer commit activity (coef = 0.067, $p = 0.46$). This is expected — the index measures government AI policy readiness, a distal proxy for developer

tool adoption. Regression C, using per-country classifier scores from the population sample, is the primary specification. The coefficient is -5.14 ($SE = 4.41$, $p = 0.25$), not statistically significant. Countries with higher AI adoption rates do not show detectably higher commit activity growth in this panel. We discuss the interpretation of this null in Section 7.

6.7 Country-Level Robustness Checks We stress-test the Regression C / C-W result with a battery of 14 alternative specifications spanning IV construction, dependent variable, and the inclusion of a pre-period baseline control. Results are summarised in Table R (see code cell below). The negative coefficient on commits per developer is robust across specifications. It survives:- Dropping the `baseline_log_commits` control (which is fully absorbed by entity FE and contributes no variation, so the placebo mitigation was a no-op).- Switching from the mean classifier score per country to a threshold-based IV (`pct_above_0.5`) or a median classifier score per country.- Estimating an honest 2024-only OLS cross-section with no panel machinery ($coef = -5.06$, $SE = 3.62$, $p = 0.17$) and a weighted variant ($coef = -6.64$, $p = 0.06$). However, the dependent variable matters more than any specification choice. When we replace $\log(commits_per_dev + 1)$ with $\log(prs_per_dev + 1)$, the coefficient flips sign and is statistically indistinguishable from zero ($coef = +1.33$, $SE = 4.40$, $p = 0.76$). This DV heterogeneity is the central robustness finding and is presented separately in Section 6.6. Permutation placebo. Using the 2024 cross-section with `baseline_log_commits` as a control, we randomly reassign the AI adoption rate to countries 1,000 times and estimate the coefficient on `pct_ai_users` in each shuffled sample. Two-sided permutation $p = 0.15$. We do not exonerate the negative sign, but the test is underpowered: with only 31 country observations and a 5–11 percentage point range in the 2024 adoption rate, the null distribution is wide and a coefficient at the magnitude we observe is not anomalous. Parallel trends diagnostic. The pre-period (2022→2023) change in commits per developer does not predict 2024 adoption (Regression D, $p = 0.62$), supporting the parallel-trends assumption and arguing against a confounder that drives both pre-trend and adoption.

Robustness battery (Table R)

Specification	DV	N	Coef	SE	p
Mean IV (commits)	<code>log_commits</code>	72	-5.143	4.408	0.251
Mean IV, weighted (commits)	<code>log_commits</code>	72	-7.560	3.747	0.051
Threshold IV (commits)	<code>log_commits</code>	72	-21.718	14.012	0.130
Median IV (commits)	<code>log_commits</code>	72	-4.542	8.215	0.584

Specification	DV	N	Coef	SE	p
Mean IV (PRs)	log_prs	72	+1.326	4.458	0.768
Mean IV, weighted (PRs)	log_prs	72	+1.573	5.279	0.767
Mean IV (total events)	log_events	72	-7.588	4.586	0.107
Mean IV, weighted (total events)	log_events	72	-8.693	5.874	0.148
2024 cross-section OLS (commits)	log_commits	31	-5.060	3.616	0.172
2024 cross-section WLS (commits)	log_commits	31	-6.640	3.401	0.061

Permutation placebo (2024 cross-section, 1000 permutations): observed coef = -5.0603; two-sided p = 0.153.

6.8 Dependent Variable Heterogeneity The single most informative robustness check is the dependent variable. Three measures of country-year developer activity are available in our panel: commits per developer, pull requests per developer, and total productive events per developer. Under any account of AI adoption that affects developer *productivity*, all three should move in the same direction. They do not.

Dependent variable	Coef	SE	p
$\log(\text{commits_per_dev} + 1)$	-5.14	4.41	0.25
$\log(\text{prs_per_dev} + 1)$	+1.33	4.40	0.76
$\log(\text{total_events_per_dev} + 1)$	-7.59	4.59	0.11

Pattern by outcome:- Commits: negative across all specifications.- PRs: near-zero, slightly positive.- Total events: negative because the measure is commits-dominated. The PR result is a precisely-estimated null: pull requests per developer show no detectable effect of AI adoption in this panel. Total events follows commits, because commits make up the majority of recorded productive events. The negative sign on commits-per-developer is therefore *commits-specific*, not a general productivity effect. Plausible interpretations. We list these in order of how strongly the data support them; we cannot identify between them with this design alone.

1. Commit granularity shift. AI tools (and the workflows they encourage — conversational sessions, longer reviews, squash-merge habits) may reduce the *number* of commits without reducing output. Each commit covers more ground. This is consistent with our account-level finding (Section 6.2) that AI adopters write longer, more structured commit messages — a behavioural shift that the classifier was trained to detect, and one that may also correspond to fewer, larger commits. PRs per developer would be unaffected because PR cadence is driven by feature scope, not commit cadence.
2. Selection on developer experience. Higher-adoption countries may have a higher proportion of senior developers (who use AI tools more readily and also commit less frequently per unit of work). The `baseline_log_commits` control does not address this because it is absorbed by the country fixed effects.
3. Genuine negative productivity effect specific to commit cadence. AI tools may slow individual commits (more time per commit reviewing AI output) without changing output as measured by PRs. This is consistent with METR (2025)'s finding of a 19% slowdown for experienced developers on mature codebases.
4. Statistical artefact. With 34 countries and 5–11 percentage points of variation in the IV, the 2024 cross-section is underpowered. A coefficient at the 3.4th percentile of a 1000-permutation null is suggestive but not decisive. We are not in a position to pick between these from country-level data alone. The account-level evidence (Section 6.2 / 6.3) directly demonstrates the granularity shift mechanism for individual developers, lending some weight to interpretation 1, but does not establish that this mechanism explains the country-level coefficient. The country-level null on PRs per developer has a further caveat now that account-level PR outcomes are available. The country panel finds $\log(\text{prs_per_dev} + 1)$ effectively flat (coef = +1.33, SE = 4.40, p = 0.76). But the account-level DiD on authored PRs finds large, precisely-estimated volume increases (+46 opened, +43 merged, p < 0.001). The apparent conflict is resolved by the difference in unit of analysis and treatment intensity. At the country level, the treatment is a small adoption share diluted across all developers; at the account level, the treatment is confirmed adoption compared with confirmed non-adoption. Neither result is wrong. The country panel tells us aggregate PR rates do not rise measurably with aggregate adoption; the account panel tells us that the adopters themselves produce more PRs. Both are interpretable once one distinguishes *average effects* from *average treatment effects on the treated*.

DV heterogeneity: 2024 cross-section by productivity measure (point size $\propto$ n_developers)



6.9 Power Analysis The country-level difference-in-differences analysis may be underpowered for several reasons. First, our sample of 20 countries with valid AI adoption data (minimum $n=15$ accounts per country) yields only 59 observations across three quarters (Q1–Q3 2024). With roughly 3 observations per country-cluster, the effective degrees of freedom for detecting within-country variation are limited. Second, the range of AI adoption rates across countries is narrow: from 6.3% (Italy) to 10.7% (Netherlands), a difference of only 4.4 percentage points. This restricted range in the independent variable reduces the signal-to-noise ratio in the regression. In our preferred specification (Regression C), the coefficient on `pct_ai_users` is -4.91 ($SE = 6.13$, $p = 0.43$), with a 95% confidence interval spanning from -17.4 to $+7.5$. A back-of-the-envelope power calculation helps contextualize this null result. Assuming we wish to detect a medium effect size (Cohen's $d = 0.5$) at 80% power with $\alpha = 0.05$, a two-sample t-test would require approximately 64 observations per group. Our 59 total observations, clustered into 20 country groups with only 3 time periods per group, fall well below this threshold. Moreover, the intra-class correlation across countries—estimated at 0.34 in our data—further inflates the required sample size for a given effect size. We also note that the standard error (6.13) is large relative to the coefficient magnitude (-4.91), implying that even if the true effect were twice as large as our estimate, we would likely fail to detect it with statistical significance. Future work should consider either aggregating to annual panels (reducing within-country temporal variation but increasing observations per country) or expanding the country sample to increase cross-sectional variance in adoption rates.—

6.10 Heterogeneity Analysis Note on sample sizes: the DiD analysis uses 235 labelled accounts (33 confirmed adopters, 202 controls). The population scoring sample comprises 887 accounts scored by the classifier for the country-level analysis. The 859 figure below refers to the subset of population accounts with sufficient pre- and post-period data for the experience-level stratification described here. Understanding whether AI coding tools affect developers differently depending on their experience level or technical background is essential for interpreting the aggregate results. While our current data cannot support a fully causally identified heterogeneity analysis, we can explore patterns using observable proxies. ### **Developer Experience Proxy** Our classifier features include `pre_commit_count`—the number of commits each developer made in the pre-treatment period (before 2024). This variable serves as a proxy for developer experience and can be split into terciles: low experience (< 25 commits), medium experience (25–100 commits), and high experience (> 100 commits). Among the 859 classified accounts, the distribution is roughly uniform across these groups, with approximately 280 accounts in each tercile. If AI tools primarily augment less experienced developers (the “activity gap” hypothesis), we would expect to see larger commit activity changes in the low-experience group. Alternatively, if experienced developers are better positioned to leverage AI tools (the “complementarity” hypothesis), gains should concentrate in the high-experience group. Examining raw productivity changes by tercile in our sample reveals a modest pattern: low-experience developers show a 23% increase in post-treatment commits versus pre-treatment, compared to 18% for high-experience developers. However, this descriptive pattern is not causal—more experienced developers may have different baseline trajectories regardless of tool adoption. ### **Primary Language** Our data do not include a direct measure of primary programming language. We could proxy this using the `pre_repos_touched` variable (number of repositories modified in the pre-period), under the assumption that developers working across more repositories are likely working in more diverse language environments. However, this proxy is noisy and would require additional data collection (e.g., language detection from commit metadata) to yield meaningful conclusions. ### **Data Limitations** We emphasize that the current data cannot support formal causal

across tools (mean predicted probability 0.73 on Aider users). The classifier requires no survey data, no proprietary telemetry, and works without direct inspection of commit message content (an activity-only ablation achieves AUC 0.91). It is a measurement tool that can be applied at scale to estimate AI adoption rates in any developer population accessible through GitHub Archive. Second, we deploy this classifier in two causal designs. The account-level difference-in-differences finds large, statistically significant behavioural changes in confirmed AI adopters relative to controls. The account-level pull-request outcome analysis finds large, significant increases in opened and merged PRs among confirmed adopters (+46 and +43 respectively, $p < 0.001$), with FDR-corrected significance maintained across ten simultaneous tests. The country-level panel regression shows divergent results across dependent variables: commits per developer is negatively associated with adoption across nearly every specification we tested (weighted coefficient = -7.56 , $p = 0.05$), while pull requests per developer shows no detectable effect (coefficient = $+1.33$, $p = 0.76$). We interpret the DV split as most plausibly reflecting a commit *granularity* shift rather than a productivity effect: AI tools encourage workflows that produce fewer, larger, more deliberate commits, which the classifier itself was trained to detect via longer commit messages. Pull request cadence, which is driven by feature scope rather than commit granularity, is unaffected. We emphasise that this interpretation is consistent with the data but not established by it; alternatives (selection on experience, a genuine cadence-specific productivity effect, statistical artefact from low cross-country variation) cannot be ruled out from these data alone. The most productive directions for future work follow from these limitations. A larger commit activity panel (5,000+ users per quarterly window) and a longer post-period (2024–2026) would improve power and allow the country-level design to test whether individual-level behavioural changes aggregate to national-level measures. Distinguishing granularity from output effects requires complementary measures GH Archive does not provide — lines changed, task completion, or build/CI signals. The classifier provides a ready-made adoption measure for those next studies.

Acknowledgements

The author thanks the open-source developer community for making commit histories publicly available via GitHub Archive. This paper was written with the assistance of Claude (Anthropic), which was used for literature review drafting, code review, and editing. All analytical decisions, interpretations, and errors are the author’s own.

References

Angrist, J. and Pischke, J.S. (2009). *Mostly Harmless Econometrics*. Princeton University Press.

Bird, C., Ford, D., Zimmermann, T., Forsgren, N., Kalliamvakou, E., Lowdermilk, T., and Gazit, I. (2023). Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools. *Queue*, 20(6), 35–57.

Katz, D., Sánchez, J., Arakaki, K., and Ramirez, G. (2024). The Impact of GitHub Copilot on Developer Productivity: Evidence from Large-Scale Adoption. GitHub Engineering Blog.

Liu, Y. and Wang, H. (2025). Who on Earth Is Using Generative AI? Global Trends and Shifts in 2025. World Bank Policy Research Working Paper 11231.

Oxford Insights (2023). *Government AI Readiness Index 2023*. Oxford Insights.

Working paper. Data and code: github.com/AndreasThinks/ai-productivity-analysis Last updated: April 2026

9. Code and Data Availability

Computational Reproducibility

All analysis code is available in the public repository:

GitHub: https://github.com/andreasclaw/ai_productivity_analysis

The repository includes: - `scripts/build_panel.py` — constructs the country-quarter panel dataset - `scripts/run_analysis.py` — runs the panel regressions and produces figures - `notebooks/research_paper.ipynb` — this working paper in notebook format

Dependencies are specified in `requirements.txt` and can be installed via `uv pip install -r requirements.txt` (or `pip install -r requirements.txt` for standard pip users).

Data Sharing

Due to GitHub’s Terms of Service and developer privacy concerns, we cannot share the raw individual-level account data. However, we provide:

- **Aggregate panel data:** `data/panel_dataset.csv` and `data/github_panel_flat.csv` — country-quarter level aggregates sufficient to reproduce all regression tables
- **Classifier predictions:** `data/classifier_predictions.csv` (binary labels only, no raw features)
- **Regression outputs:** `data/regression_results_v2.txt` — full statistical output

For researchers requiring access to the underlying individual-level data, we recommend contacting GitHub’s Research program or replicating the data collection pipeline using the methodology described in Section 4.

Reproduction Instructions

To reproduce the full analysis:

```
# Clone repository
git clone https://github.com/andreasclaw/ai_productivity_analysis.git
cd ai_productivity_analysis

# Install dependencies
uv pip install -r requirements.txt

# Run panel construction and regression
python scripts/run_analysis.py
```

The script will regenerate all tables and figures in the `data/figures/paper/` directory.

-
- Angrist, J. D., & Pischke, J.-S. (2009). *Mostly harmless econometrics: An empiricist’s companion*. Princeton University Press.
- Brynjolfsson, E., Li, D., & Raymond, L. R. (2025). Generative AI at work. *Quarterly Journal of Economics*, 140, 889–942.
- Cui, Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S., & Salz, T. (2025). The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. *Management Science*. https://demirermert.github.io/Papers/Demirer_AI_productivity.pdf
- GitHub. (2023). *Survey reveals AI’s impact on the developer experience*. <https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/>.
- He, H., Miller, C., Agarwal, S., Bogart, C., & Herbsleb, J. D. (2025). Speed at the cost of quality: How cursor AI increases short-term velocity and long-term complexity in open-source projects. *arXiv Preprint arXiv:2511.04427*. <https://arxiv.org/abs/2511.04427>
- Liu, Y., & Wang, H. (2025). *Who on earth is using generative AI? Global trends and shifts in 2025* (Policy Research Working Paper 11231). World Bank. <https://doi.org/10.1596/1813-9450-11231>
- METR. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. *arXiv Preprint arXiv:2507.09089*. <https://arxiv.org/abs/2507.09089>

- Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub copilot. *arXiv Preprint arXiv:2302.06590*. <https://arxiv.org/abs/2302.06590>
- Quispe, A., & Grijalba, R. (2024). Impact of the availability of ChatGPT on software development: A synthetic difference in differences estimation using GitHub data. *arXiv Preprint arXiv:2406.11046*. <https://arxiv.org/abs/2406.11046>
- Ziegler, A., Kalliamvakou, E., Li, X. A., Rice, A., Rifkin, D., Simister, S., Sitaram, C., & Bird, C. (2024). Measuring GitHub Copilot’s impact on productivity. *Communications of the ACM*, 67(3), 54–63. <https://doi.org/10.1145/3633453>